

the URL the anchor points to, you can use `::attr(href)`.

Let's see the interaction with the scrapy shell when we try our new selectors:

```
>>> response.selector.css('h1>a::text').
extract()
['\n  Explore GitHub\n  ']
>>> response.selector.css('h1>a::attr(href)').
extract()
['/explore']
```

We can see that we are now getting the heading of the page, plus some extra whitespace. We are also getting the URL that the heading points to.

While we just used CSS here to select content, in reality the selectors are being translated to XPath. We can also use XPath directly when that makes more sense. You can accomplish the same extract of the page heading using XPath as follows:

```
>>> response.selector.xpath('//h1/a/text()').
extract()
['\n  Explore GitHub\n  ']
>>> response.selector.xpath('//h1/a/@
href').extract()
['/explore']
```

XPath is almost like a file path, where `h1/a` means the `a` tag inside the `h1` tag. The double-slash in the beginning tells it to search for all `h1` tags. Finally `text()` returns the text inside a node and `@attr` returns at attribute names `attr`. We won't go any deeper into XPath in this article, but it is worth learning if you're interested in scraping. Even with the knowledge of XPath you should prefer the CSS syntax when dealing with things like CSS classes since the XPath alternative to that is quite complex.

Scrapy also includes a reselector that uses regular expressions, which you can use to extract parts of your final selection.

A very powerful and useful feature of selectors is that they can be chained. For instance you can use a selector to match a table, store that in a variable (without using `extract`) and then further apply selectors to that subsection of the selection.

You can keep using the shell to explore the page, but it can be hard to get to the relevant content through blind exploration. It's wise to open a browser in parallel, and use tools in the browser to see how the content is structured.

Exploring using the Browser

The browser is a powerful tool to explore a page before you start scraping. You can use web inspection tools in most modern

*pointers

>>> Interesting Hubot developer videos



*Scraping web pages

>>> Watch a simple tutorial about writing a crawler using Scrapy to scrape and parse a webpage.

<http://dgit.in/WebScrapy>



*Scraping Reddit

>> Always wanted to extract the web content of Reddit? Here's a complete tutorial.

<http://dgit.in/ScrapingReddit>



*Scrapy Cloud

>>> Quick tutorial on how to deploy a Scrapy web crawler to the Scrapy Cloud platform.

<http://dgit.in/ScrapyCloud>



*Scrapy series

>>> A basic video tutorial series of using the Python Scrapy framework through the Windows command prompt.

<http://dgit.in/ScrapyTut>



browsers to know just what kind of selectors you can use to get to the data you want.

But beware, the HTML you see in the web inspector might not be what you see in Scrapy! Many modern websites use generous amounts of JavaScript to modify the web page after it is loaded, and these modifications will be absent in Scrapy since it just loads the page as text.

If you see a discrepancy between the browser tools and Scrapy, chances are the page's code was modified by JavaScript. If something is entirely missing from the page in Scrapy but present on the site, chances are it was loaded separately, and you might need to find out that URL and scrape that too.

Take advantage of the 'view source' functionality (usually `Ctrl+u`) to see the unaltered original HTML content of the page. You can also use `view(response)` in the shell to open the page as Scrapy sees it in a browser.

Start by exploring `https://github.com/` trending in your browser and try to figure out the selectors for yourself.

Implementing our Scraper

Let's start with something simple, the `Item` class. It's open `items.py` and make sure the `GithubItem` class looks like this:

```
class GithubItem(scrapy.Item):
    repo = scrapy.Field()
    language = scrapy.Field()
    stars = scrapy.Field()
    forks = scrapy.Field()
```

Now let's open the `trending.py` file and have a look at our spider:

```
class TrendingSpider(scrapy.Spider):
    name = "trending"
    allowed_domains = ["github.com"]
    start_urls = ['https://github.com/
trending']
```

```
def parse(self, response):
    pass
```

The name here is a unique name for your scraper for command line usage. The `allowed_domains` restricts crawling to this domain, but if you want to enforce that you need to change some settings. Finally, `start_urls` gives Scrapy the URLs to start the scraping job with. Only the `name` here is required, instead of `start_urls` you can create a `start_requests` method that generates the starting requests dynamically.

When Scrapy runs your spider, it will fetch the URLs one by one and send the